

# BX GitHub Repositories

---

## Manual for Shop Operators

### Purpose of this guide

This guide describes how to set up the BX GitHub Repositories module for production use.

After completing these steps, your shop can:

- access its own GitHub repositories,
  - retrieve release ZIP files,
  - store ZIP files in the shop's **download** folder,
  - and use only the repositories approved for that specific shop.
- 

### Result after setup

The following three values must be configured successfully:

1. GitHub App ID
2. Installation ID
3. Private Key (PEM)

Additionally:

- The connection test in the module is successful.
  - Repositories of the installation can be loaded and selected.
- 

### Prerequisites

1. A GitHub account with permission to create and install a GitHub App.
  2. Admin access to the shop.
  3. Write permissions for the shop folder **download**.
  4. Access to the repositories that will be synchronized.
- 

### Step 1: Create the GitHub App

1. In GitHub, click your profile icon in the top right corner.
2. Open **Settings**.
3. Open **Developer settings** in the left menu.
4. Select **GitHub Apps**.
5. Click **New GitHub App**.
6. Enter a unique name, for example **shopname-release-sync**.
7. Enter the **Homepage URL** (for example, your shop URL).
8. Set a **Callback URL** only if user login via GitHub is required. Not relevant for this use case.
9. If webhooks are not needed: disable webhooks. They are not required for pure version synchronization.

10. Set minimal permissions:

- Repository permission **Metadata: Read-only**
- Repository permission **Contents: Read-only**

11. Define where the app can be installed.

12. Save the app.

Note: Grant as few permissions as possible (least privilege).

---

## Step 2: Install the GitHub App

1. Open **Install App** in your GitHub App settings.
2. Select the target account or organization.
3. Set repository access to **Selected repositories**.
4. Select only the repositories your shop should use.
5. Confirm the installation.

Important: This selection determines which repositories the shop can technically access.

---

## Step 3: Retrieve the App ID

1. Open the GitHub App settings.
  2. Copy the **App ID**.
  3. Save this value.
- 

## Step 4: Retrieve the Installation ID

1. Open the app installation view.
2. Read the installation ID from the URL or from the installation details.
3. Save this value.

Example: If the URL contains `.../settings/installations/12345678`, then **12345678** is the Installation ID.

---

## Step 5: Generate the private key (PEM)

1. In the GitHub App configuration, open **Private keys**.
2. Click **Generate a private key**.
3. Store the downloaded PEM file securely.
4. Keep the PEM file ready for module configuration.

Important:

- Never send private keys by email.
  - Never post private keys in tickets or chats.
  - If you suspect a leak, generate a new key immediately and revoke the old one.
-

## Step 6: Enter data in the shop module

1. Open the BX GitHub Repositories module in the shop admin.
  2. Fill in the following fields:
    - **GitHub App ID**
    - **Installation ID**
    - Upload **Private Key (PEM)** file
  3. Run **Test connection**.
  4. On success, click **Save settings**.
- 

## Step 7: Create and configure the template product

Before the first download products can be created, a template product must exist in the shop.

1. Create a new product in the desired target category in the shop admin.
2. Maintain this product explicitly as a template and do not use it as a regular sellable product.
3. Set the template product to **inactive** and keep it inactive permanently.
4. Set the **sort order** of the template product to **9999**.
5. Maintain all default values that should later be inherited by automatically created products.
6. Assign a download entry to the template product in the product option values **Downloads**.
7. Deliberately assign the file **dummy.zip** as the download file for this template.
8. Particularly important are the category assignment, tax class, and any other desired default product settings.
9. Note the **products\_id** of this template product.
10. Store this ID in the module settings as **Template Product ID**.

Important:

- The template product is only a blueprint and always remains inactive.
  - Newly created repository products are copied from this template.
  - The first download attribute of the template product is copied as well.
  - The assigned file **dummy.zip** is automatically replaced in the created repository product with the filename of the respective repository ZIP.
  - The template defines the category assignment and many other default parameters of the automatically created products.
- 

## Step 8: Load repository list and set selection

1. After a successful connection test, load the repository list.
2. Select the required repositories and save the selection.
3. Download repositories.

Import target: The ZIP files are stored in the shop folder **download**.

Important: Each repository must have at least one version tag (for example **v2.0.5**). Only then can the module download a ZIP file. Without a version tag, ZIP download is not possible.

Example:

```
git tag -a v2.0.5 -m "BX Products Video v2.0.5"  
git push origin v2.0.5
```

---

## Typical errors and solutions

### Error 401 / 403 during connection test

Possible cause:

- App ID, Installation ID, and PEM do not belong together.
- The app is not installed for the target repository.
- Permissions are too restrictive.

Solution:

1. Verify all values again.
2. Verify the installation.
3. Verify permissions for your use case.

### Repository list is empty

Possible cause:

- No repositories are enabled in the installation.
- Wrong account or wrong organization selected.

Solution:

1. Open the installation in GitHub.
2. Check **Selected repositories**.
3. Grant access to the required repositories.

### ZIP import does not work

Possible cause:

- Release asset does not match the expected pattern.
- Missing write permissions in the **download** folder.

Solution:

1. Check asset name and pattern.
2. Check write permissions for **download**.
3. Run the test again.

---

## Security checklist before go-live

1. Only minimal permissions are enabled.

2. Installation is set to **Selected repositories**.
  3. No sensitive data is stored in logs in plain text.
  4. Private key is stored securely.
  5. Test run is successful.
  6. Download file is correctly written to the **download** folder.
  7. Scheduled task execution is enabled.
- 

## Maintenance and rotation

1. Rotate private keys regularly.
2. Remove repositories no longer needed from the installation.
3. Run the connection test again after permission changes.
4. Review error logs regularly.